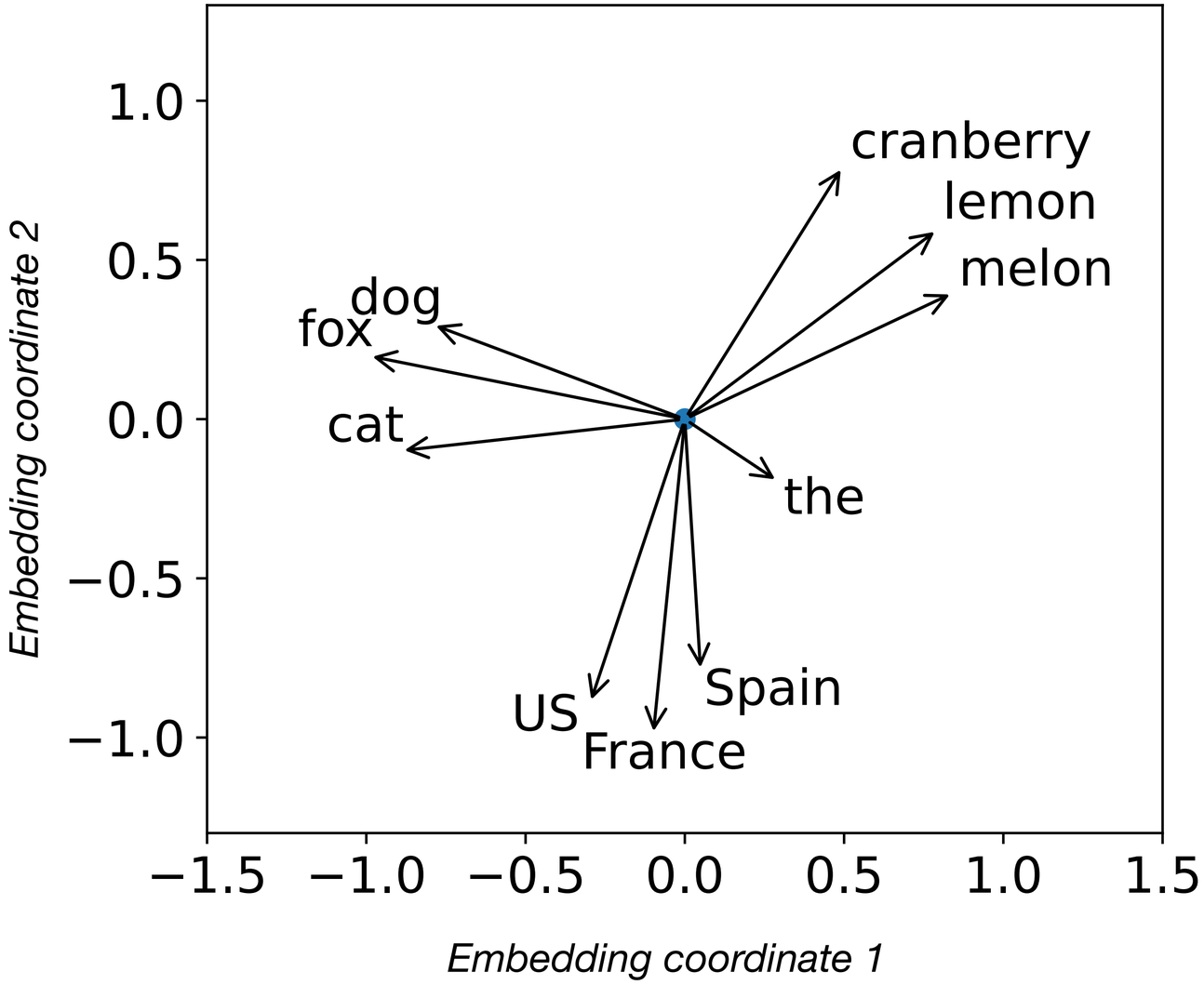
Vector-space embeddings

10-701 Introduction to Machine Learning

Geoff Gordon and Pradeep Ravikumar

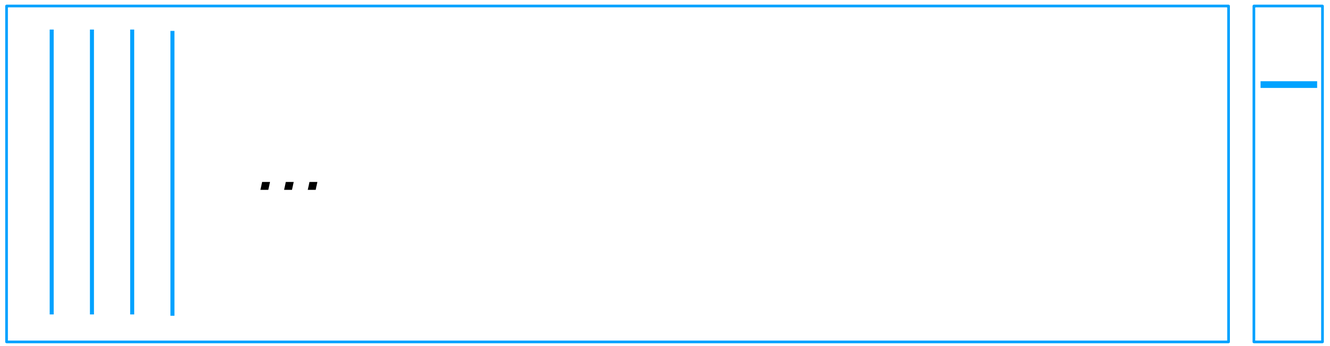
Vector-space embedding



Arrows = embeddings v_j (here, in $\mathbb{R}^2$)

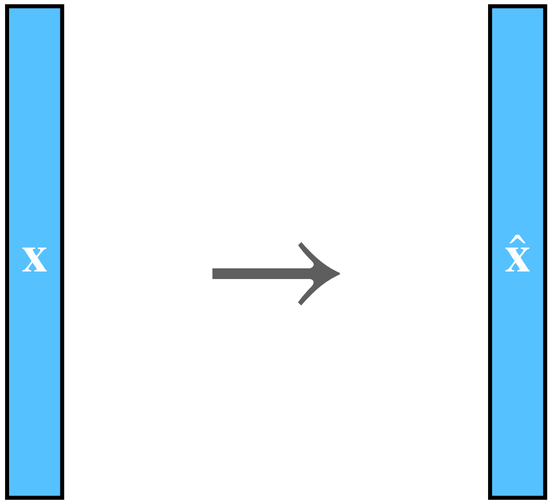
Where do embeddings come from?

$$W_1 e_{\text{dog}} =$$



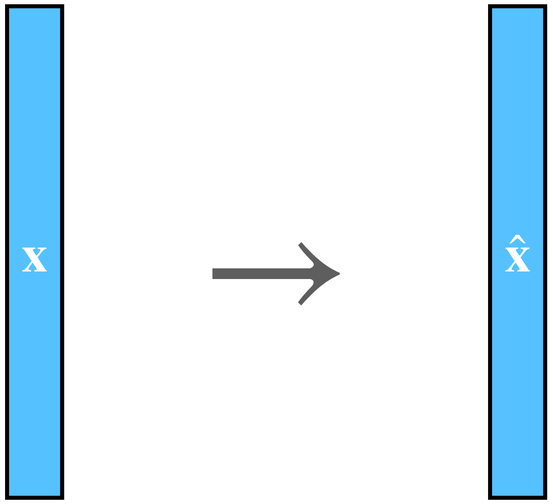
- If we present tokens as one-hot in *any* task, first layer automatically learns a vector-space encoding
- But we can often improve embeddings w/ a ***surrogate*** or ***proxy*** task
  - a task we don't directly care about, but which is related enough to help learning the task we do care about, and for which we can easily/cheaply get lots of training data

Proxy task: autoencoder



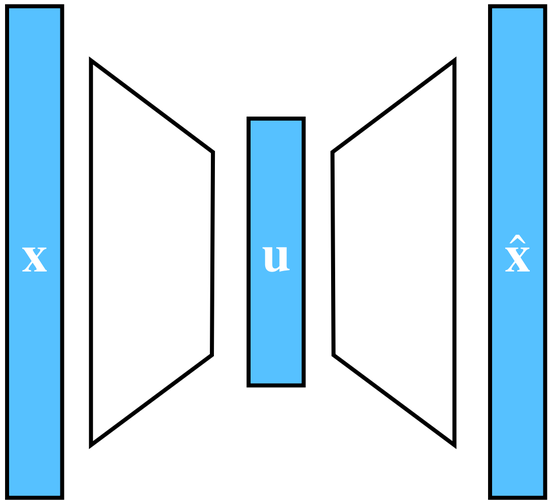
- Predict a sequence of tokens $\mathbf{x}^{(i)}$ from input $\mathbf{x}^{(i)}$
  -

Proxy task: autoencoder



- Predict a sequence of tokens $\mathbf{x}^{(i)}$ from input $\mathbf{x}^{(i)}$
  - seems kind of easy

Proxy task: autoencoder



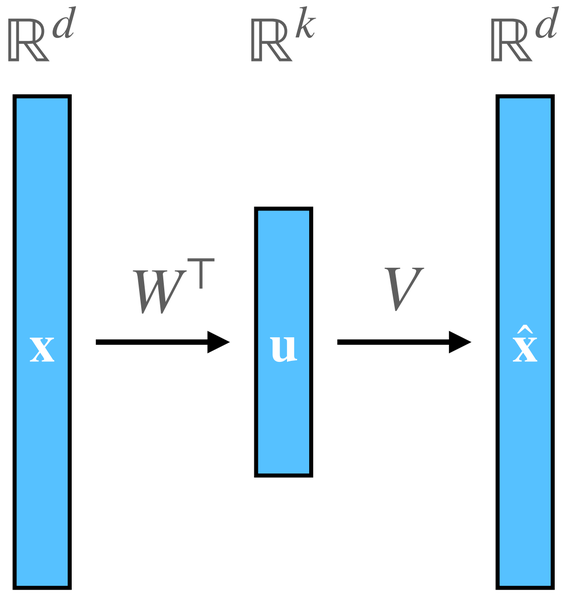
Why the name? "Code" = hidden activations, so $\mathbf{x}$ encodes (and then decodes) itself

- Predict a sequence of tokens $\mathbf{x}^{(i)}$ from input $\mathbf{x}^{(i)}$
  - seems kind of easy
- The catch: something about the model (the ***bottleneck***) prevents us from just copying input to output
  - e.g., hidden layer $\mathbf{u}^{(i)}$ w/ too few dimensions
  - e.g., regularizer that disfavors straight copying

Why auto-encoding?

- Autoencoding is a good proxy task because:
  - it's related to many other tasks: the hidden vector $\mathbf{u}$ has to be a decent summary of the token string
  - it's really easy to get training data: any string of tokens is fair game (a novel, a blog post, a product review, a newspaper article)
- Hyperparameter: granularity
  - could pass in an entire newspaper article, or split it into paragraphs, or take smaller fixed-length windows of tokens

Linear autoencoder



$$\hat{\mathbf{x}} = V\mathbf{u} = VW^\top\mathbf{x}$$

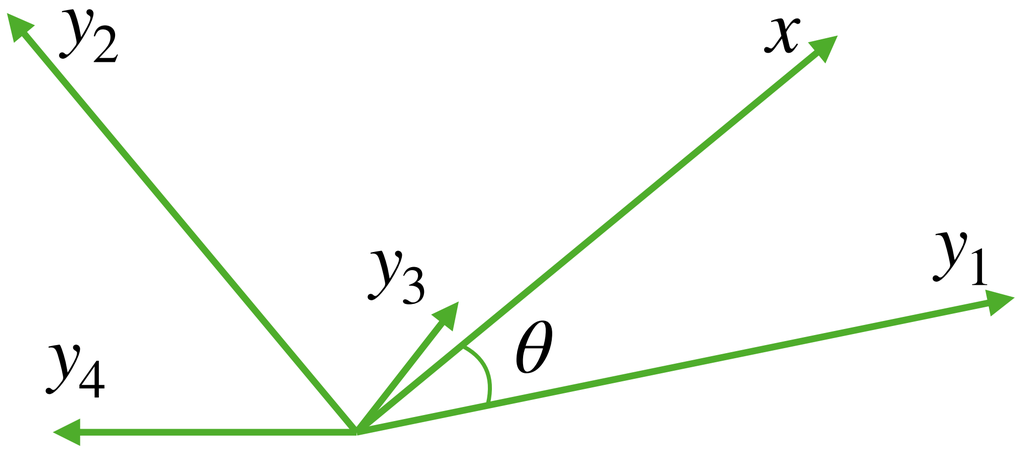
$$V, W \in \mathbb{R}^{d\times k}$$

$$k \ll d$$

- Simplest autoencoder: one hidden layer, no nonlinearities
  - min sum squared error: $\min_{V,W} \sum_{i=1}^{N} \|VW^\top\mathbf{x}^{(i)} - \mathbf{x}^{(i)}\|^2$
  - called *reconstruction error*
  - can solve e.g. w/ SGD or linear algebra

*Linear autoencoders are strongly related to the **principal components analysis** model (covered soon)*

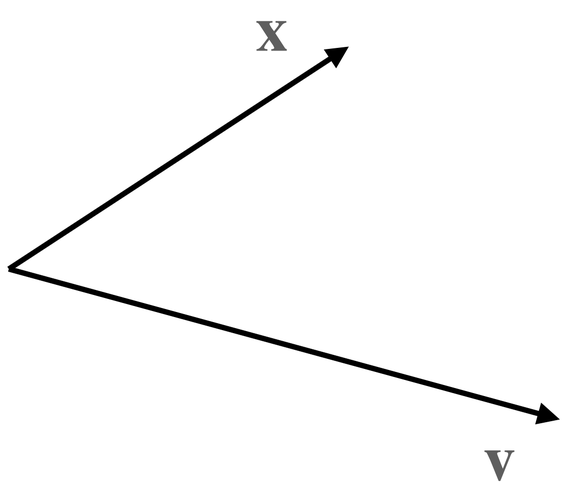
Reminder: dot product



- Dot product $x \cdot y$ depends on both *norm* and *direction* of input vectors x and y
  - $x \cdot y = \|x\| \, \|y\| \cos\theta$
- Large dot product means
  - x and y point in in approximately the *same direction*
  - x and y both have *large norm*

Reminder: vector projection

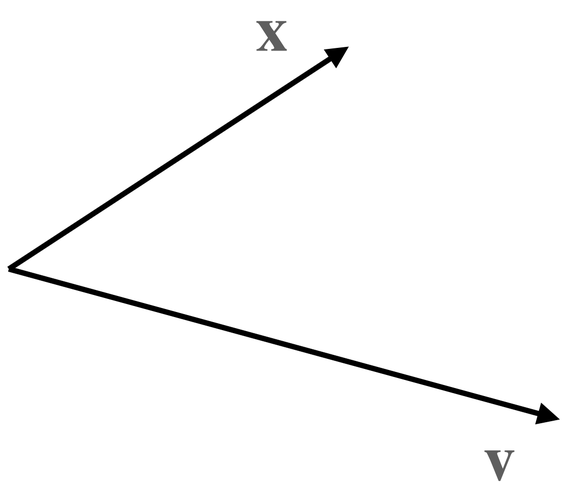
- Projection of $\mathbf{x}$ onto $\mathbf{v}$: best multiple $\lambda\mathbf{v}$ that minimizes distance (or squared distance) $\|\mathbf{x} - \lambda\mathbf{v}\|^2$
- Length of projection of $\mathbf{x}$ onto $\mathbf{v}$:
  - $a = \mathbf{v}^\top\mathbf{x}$ if $\|\mathbf{v}\| = 1$



- Projection of $\mathbf{x}$ onto $\mathbf{v}$:
  - $\hat{\mathbf{x}} = \mathbf{v}(\mathbf{v}^\top\mathbf{x})$ if $\|\mathbf{v}\| = 1$

Reminder: vector projection

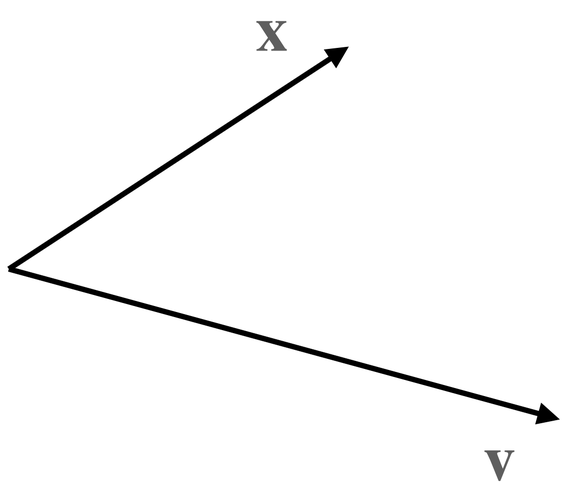
- Projection of $\mathbf{x}$ onto $\mathbf{v}$: best multiple $\lambda\mathbf{v}$ that minimizes distance (or squared distance) $\|\mathbf{x} - \lambda\mathbf{v}\|^2$
- Length of projection of $\mathbf{x}$ onto $\mathbf{v}$:
  - $a = \dfrac{\mathbf{v}^\top\mathbf{x}}{\|\mathbf{v}\|}$ if $\|\mathbf{v}\| = 1$ o/w



- Projection of $\mathbf{x}$ onto $\mathbf{v}$:
  - $\hat{\mathbf{x}} = \mathbf{v}(\mathbf{v}^\top\mathbf{x})$ if $\|\mathbf{v}\| = 1$

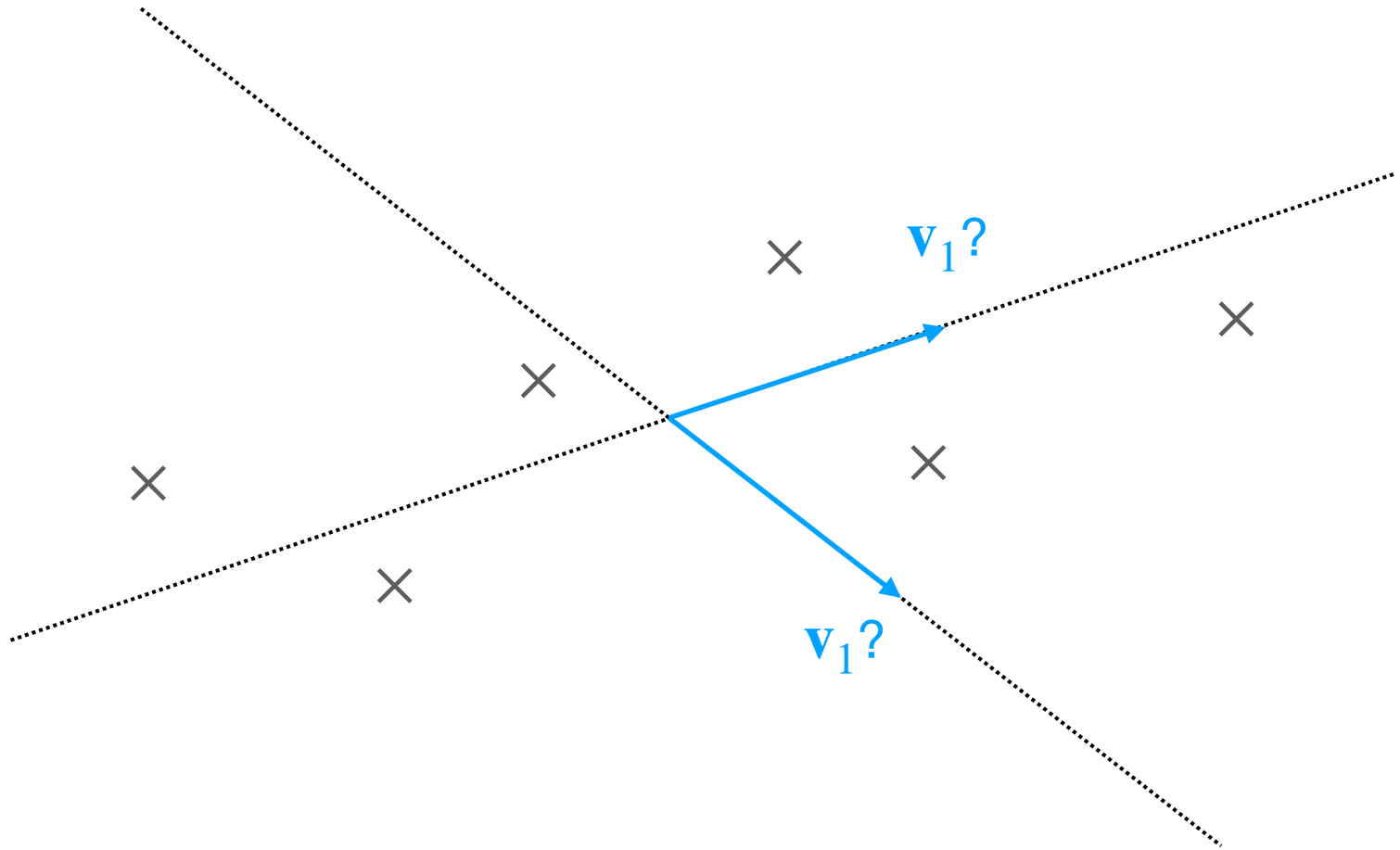
Reminder: vector projection

- Projection of $\mathbf{x}$ onto $\mathbf{v}$: best multiple $\lambda\mathbf{v}$ that minimizes distance (or squared distance) $\|\mathbf{x} - \lambda\mathbf{v}\|^2$
- Length of projection of $\mathbf{x}$ onto $\mathbf{v}$:
  - $a = \dfrac{\mathbf{v}^\top\mathbf{x}}{\|\mathbf{v}\|}$ if $\|\mathbf{v}\| = 1$, o/w



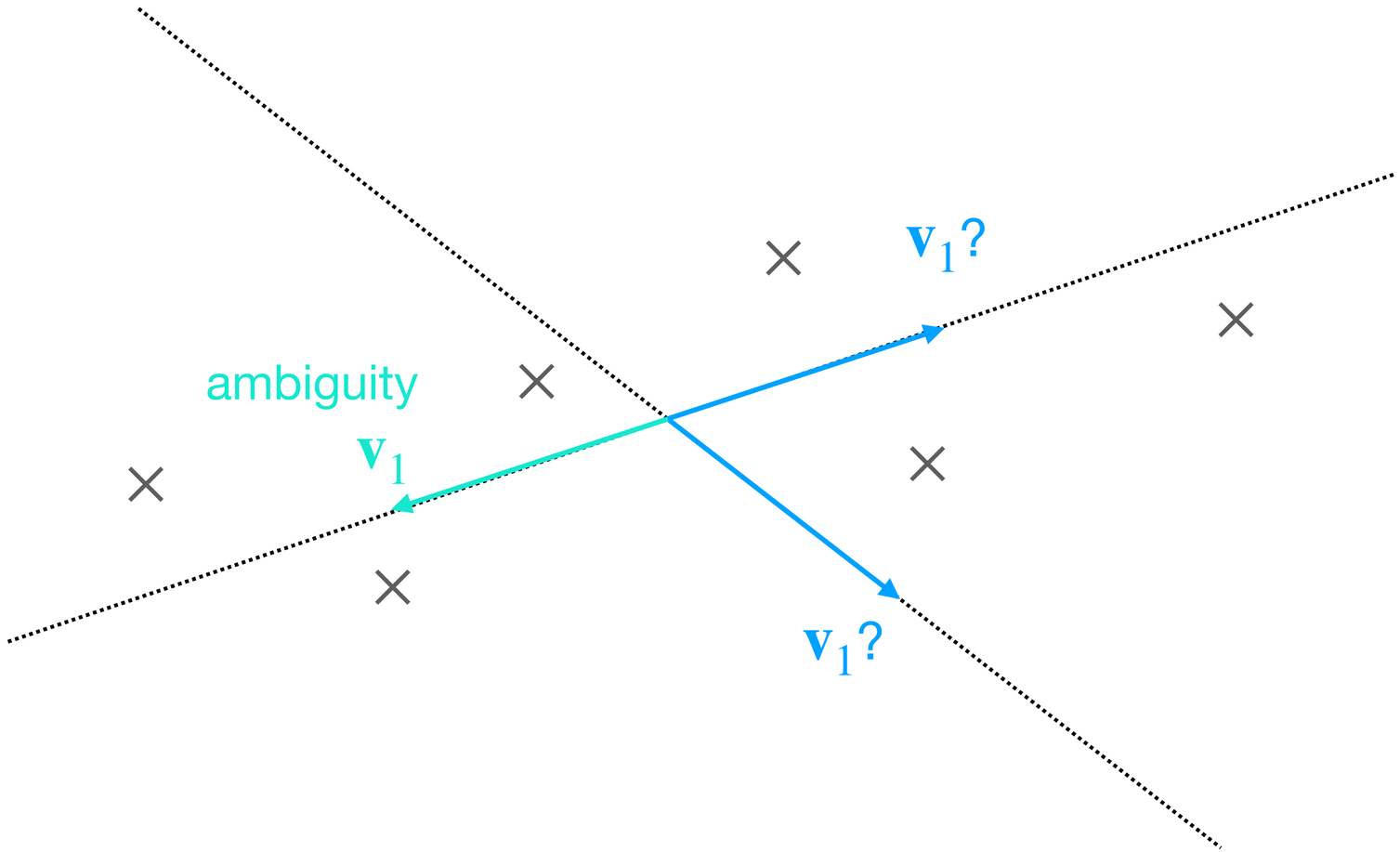
- Projection of $\mathbf{x}$ onto $\mathbf{v}$:
  - $\hat{\mathbf{x}} = \dfrac{\mathbf{v}(\mathbf{v}^\top\mathbf{x})}{\|\mathbf{v}\|^2}$ if $\|\mathbf{v}\| = 1$, o/w

1-d autoencoder



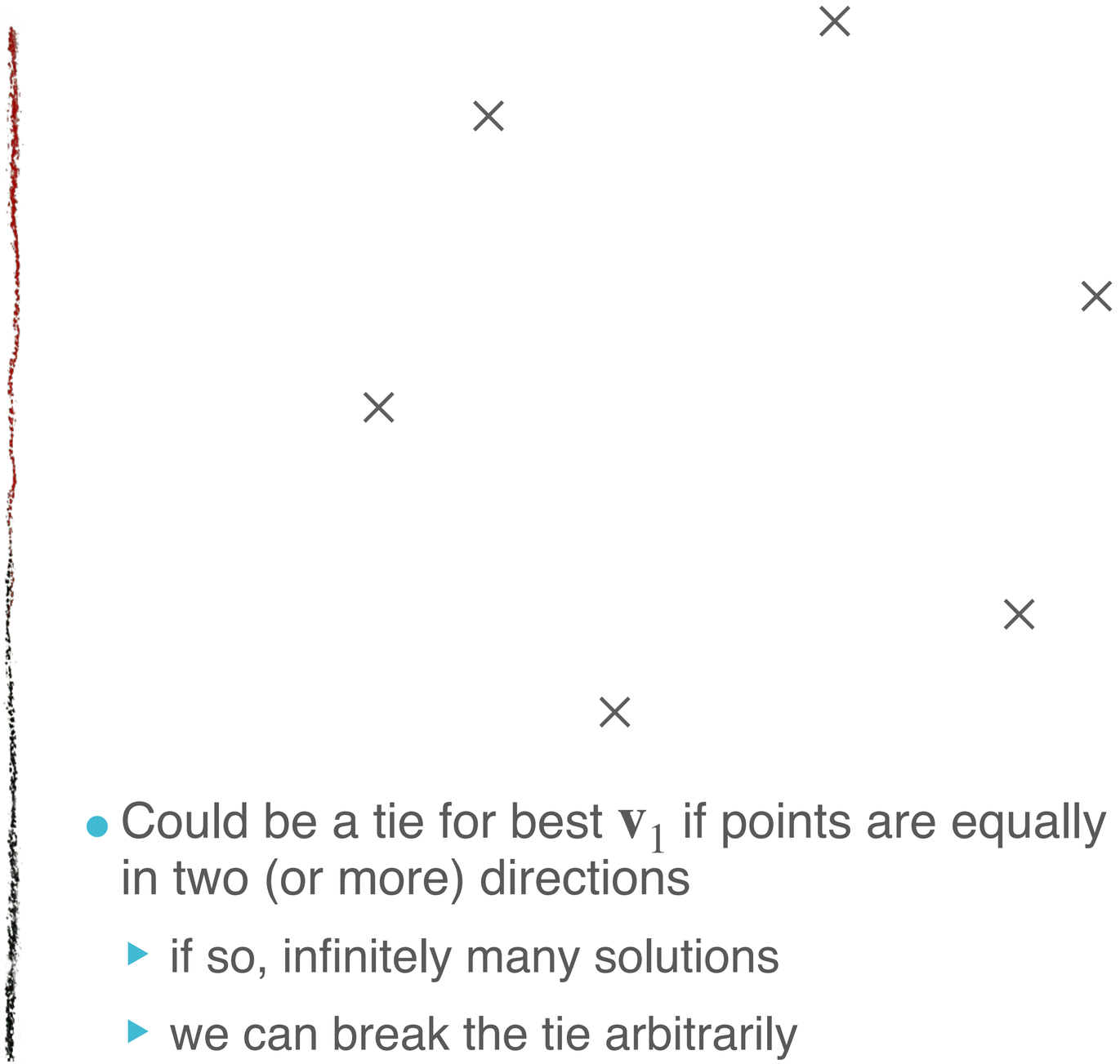
- Consider 1-d autoencoder: find $\mathbf{v}_1$ by min sum-squared reconstruction error
  - intuitively: if datapoints are spread out, $\mathbf{v}_1$ should point along the long direction

1-d autoencoder



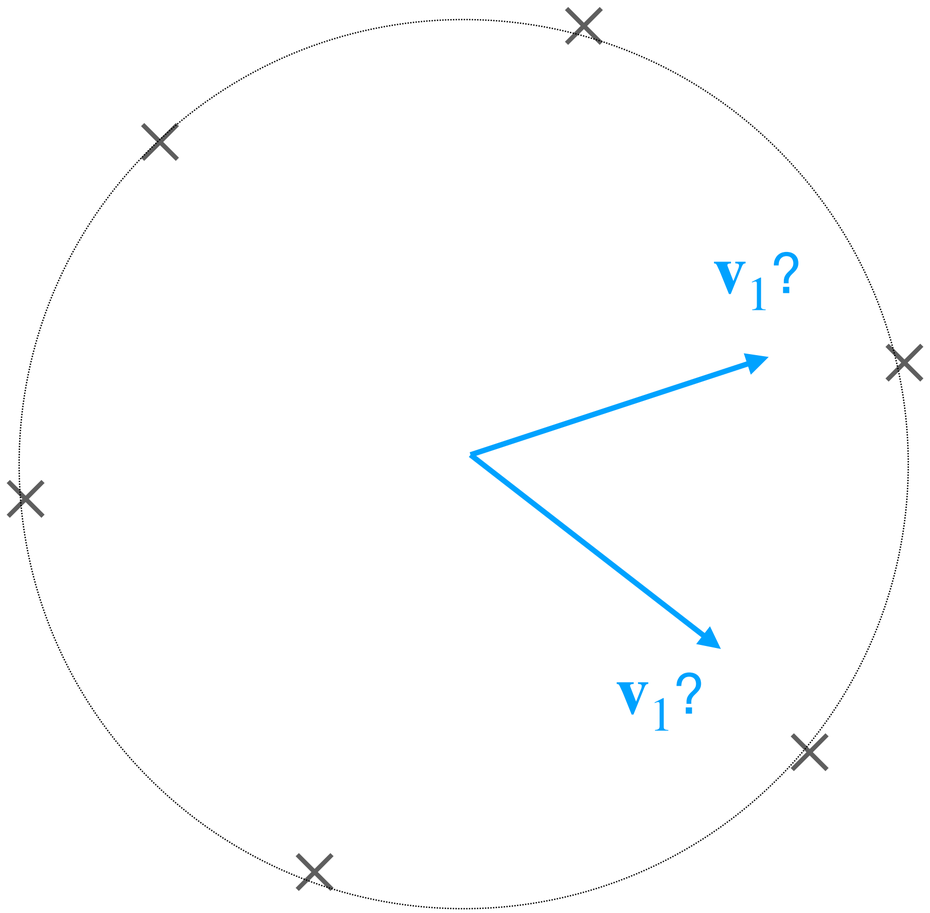
- Consider 1-d autoencoder: find $\mathbf{v}_1$ by min sum-squared reconstruction error
  - intuitively: if datapoints are spread out, $\mathbf{v}_1$ should point along the long direction

Ambiguity



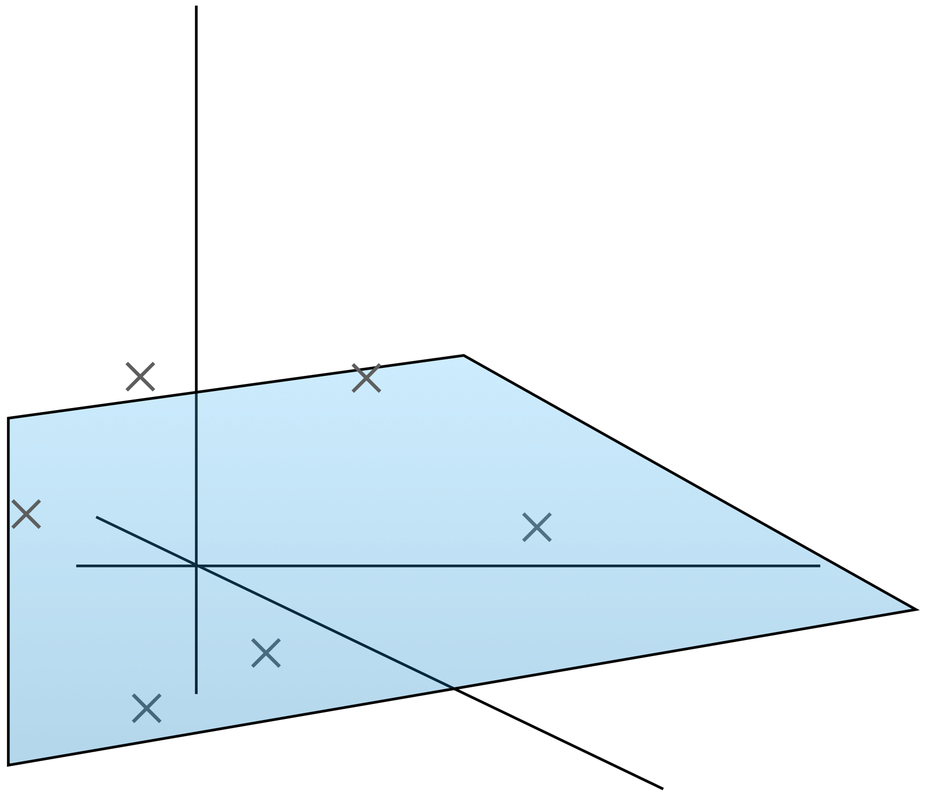
- Could be a tie for best $\mathbf{v}_1$ if points are equally spread out in two (or more) directions
  - if so, infinitely many solutions
  - we can break the tie arbitrarily

Ambiguity



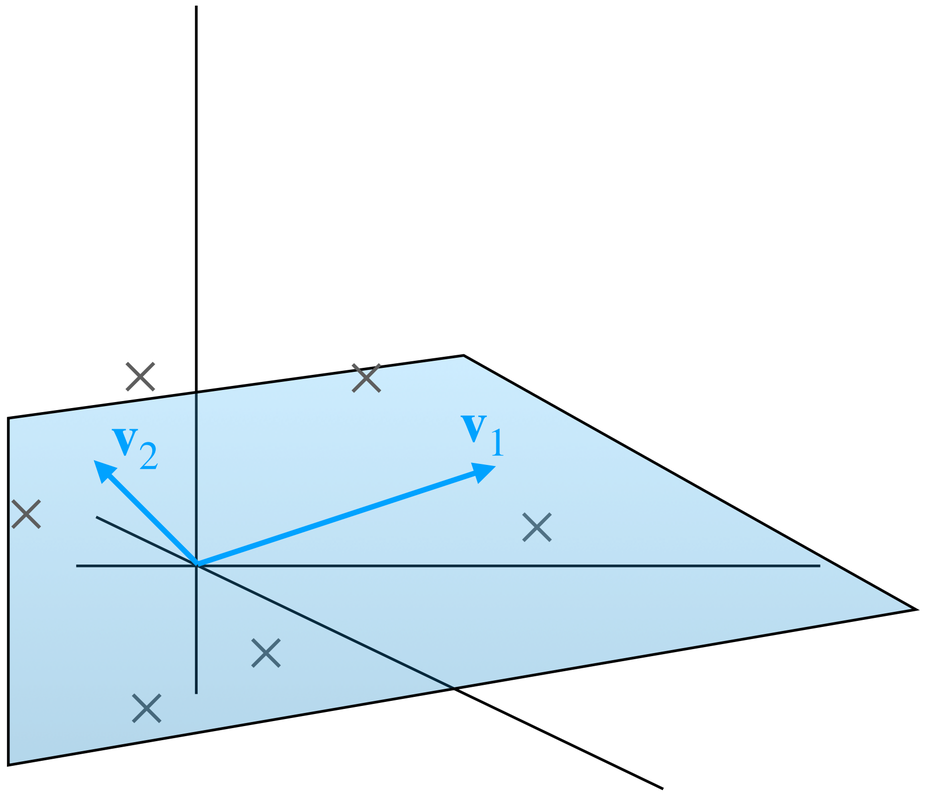
- Could be a tie for best $\mathbf{v}_1$ if points are equally spread out in two (or more) directions
  - if so, infinitely many solutions
  - we can break the tie arbitrarily

Best plane



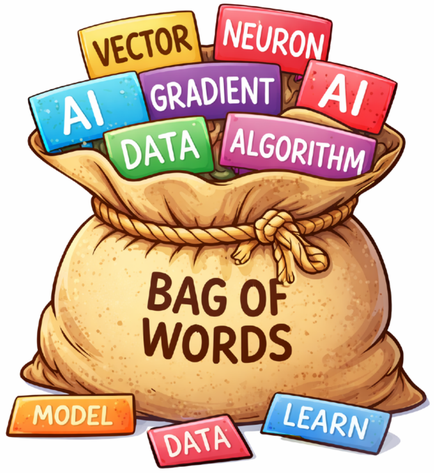
- What about $k = 2$?
  - project onto best 2D plane (min squared error)

Ambiguity: basis



- Can choose any basis for the plane — reconstruction (and reconstruction error) will be the same

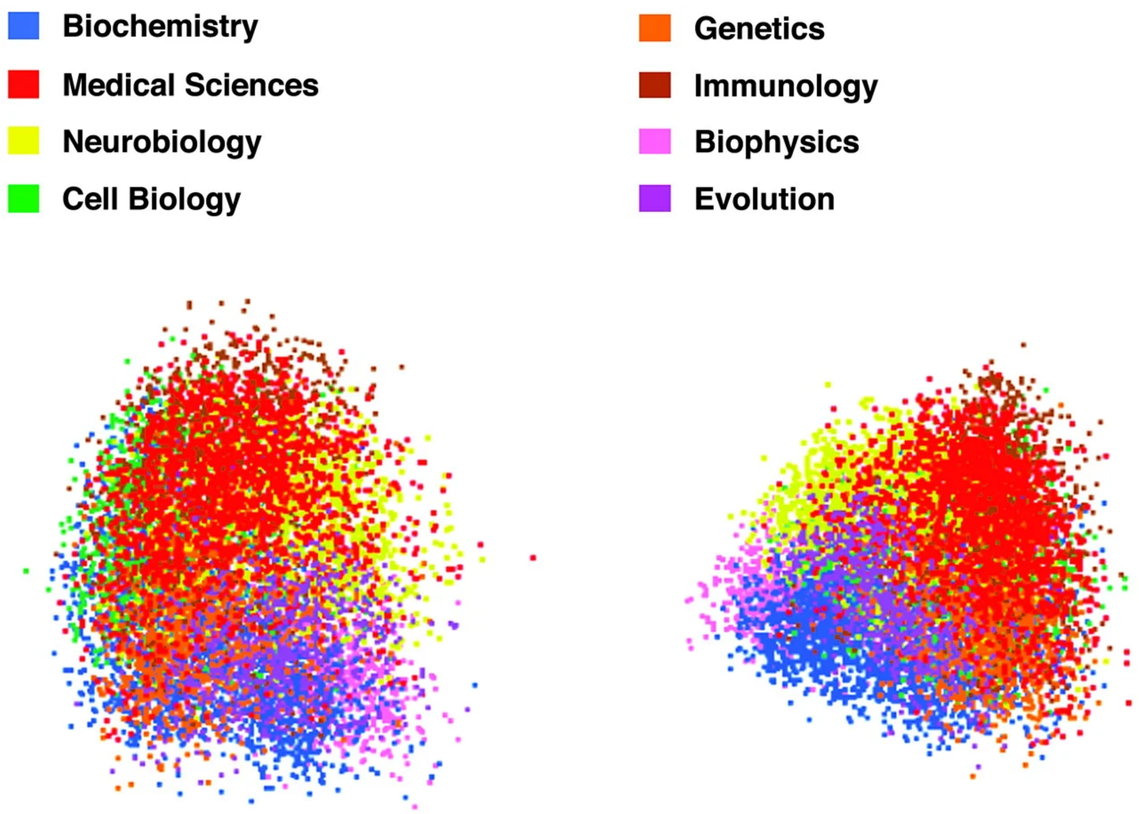
Bag of words



- One way to get vector-space embeddings: fit a linear autoencoder to a dataset of strings of tokens
- How can we pass a token string to a linear model?
  - simplest way: sum up all of the tokens
  - "quick brown fox" $\rightarrow e_{\text{quick}} + e_{\text{brown}} + e_{\text{fox}}$
  - permutation-invariant (order doesn't matter)
  - i.e., a ***multiset*** or ***bag*** of words
- BoW assumption is only to make model simpler: "man bites dog" is different from "dog bites man"

Landauer et al., PNAS 2004 https://www.pnas.org/doi/10.1073/pnas.0400341101

LSA



- This model (linear autoencoder on bags of words) is ***latent semantic analysis***
  - details: stopwords, IDF weighting, no mean subtraction
  - ex: LSA of articles from PNAS shows that article embeddings reveal broad topic

Proxy task: masked token prediction

masked token prediction

The quick brown fox [???] over the lazy dog

- Masked token prediction:
  - pick a token index i
  - *input:* a window of tokens near index i but ***not*** token i: $j \in \{i-k, \ldots, i+k\} \setminus \{i\}$ (the ***context***)
  - could also mask more than 1 token in a context window
  - *target:* the masked token i

Word2vec

masked token prediction

The quick brown fox [???] over the lazy dog

- Early successful *nonlinear* embedding learner
- Train embedding matrix V by masked token prediction
- Two layer network:
  - first layer: embed and sum tokens in context

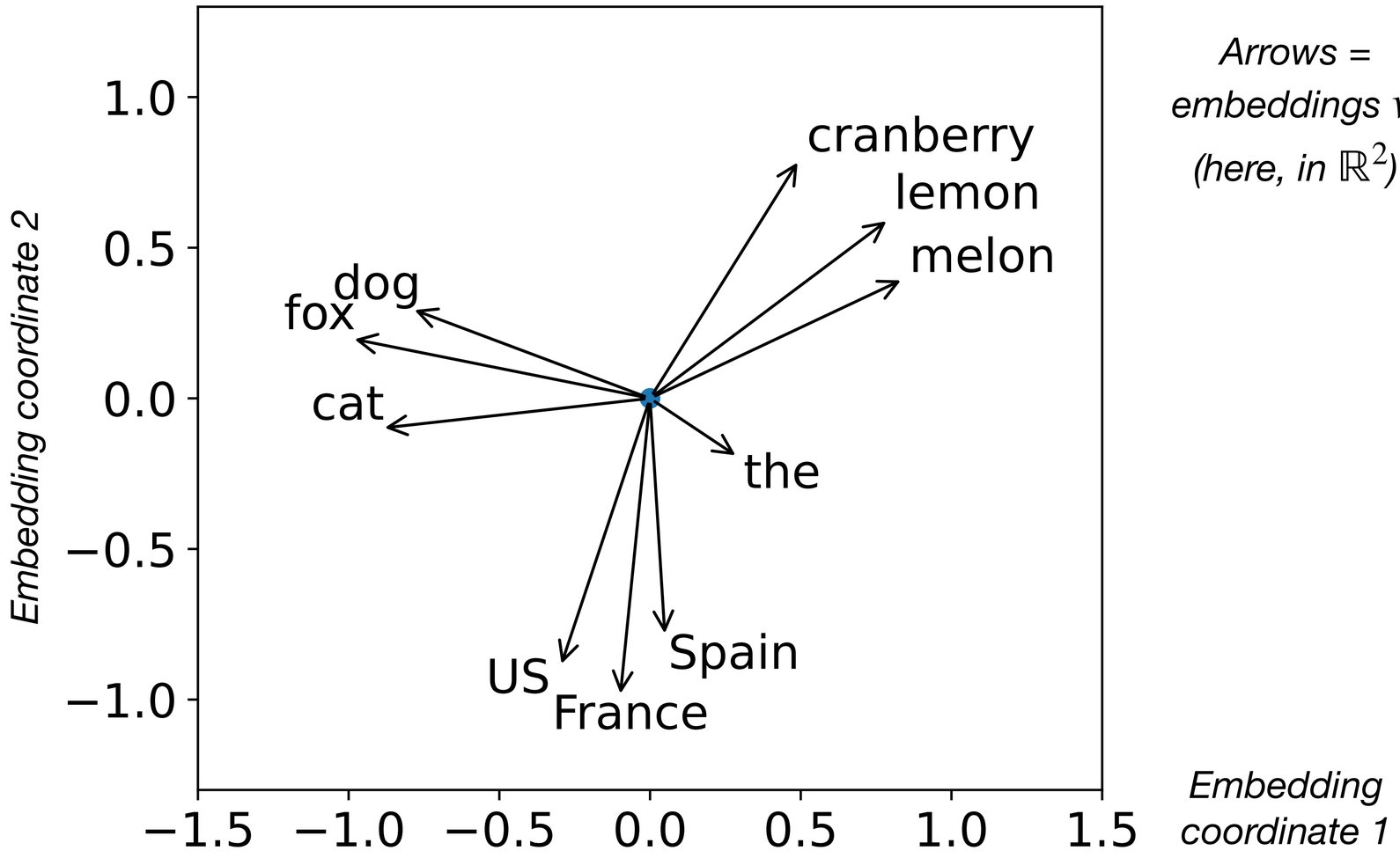
$$z = \sum_{j=i-k,\, j\neq 0}^{i+k} V x_j$$

  - second layer: search over vocabulary

$$P(y \mid \text{input}) = \frac{\exp(z \cdot Vy)}{\sum_{y'} \exp(z \cdot Vy')}$$

  - note: approximations to make it fast

First layer

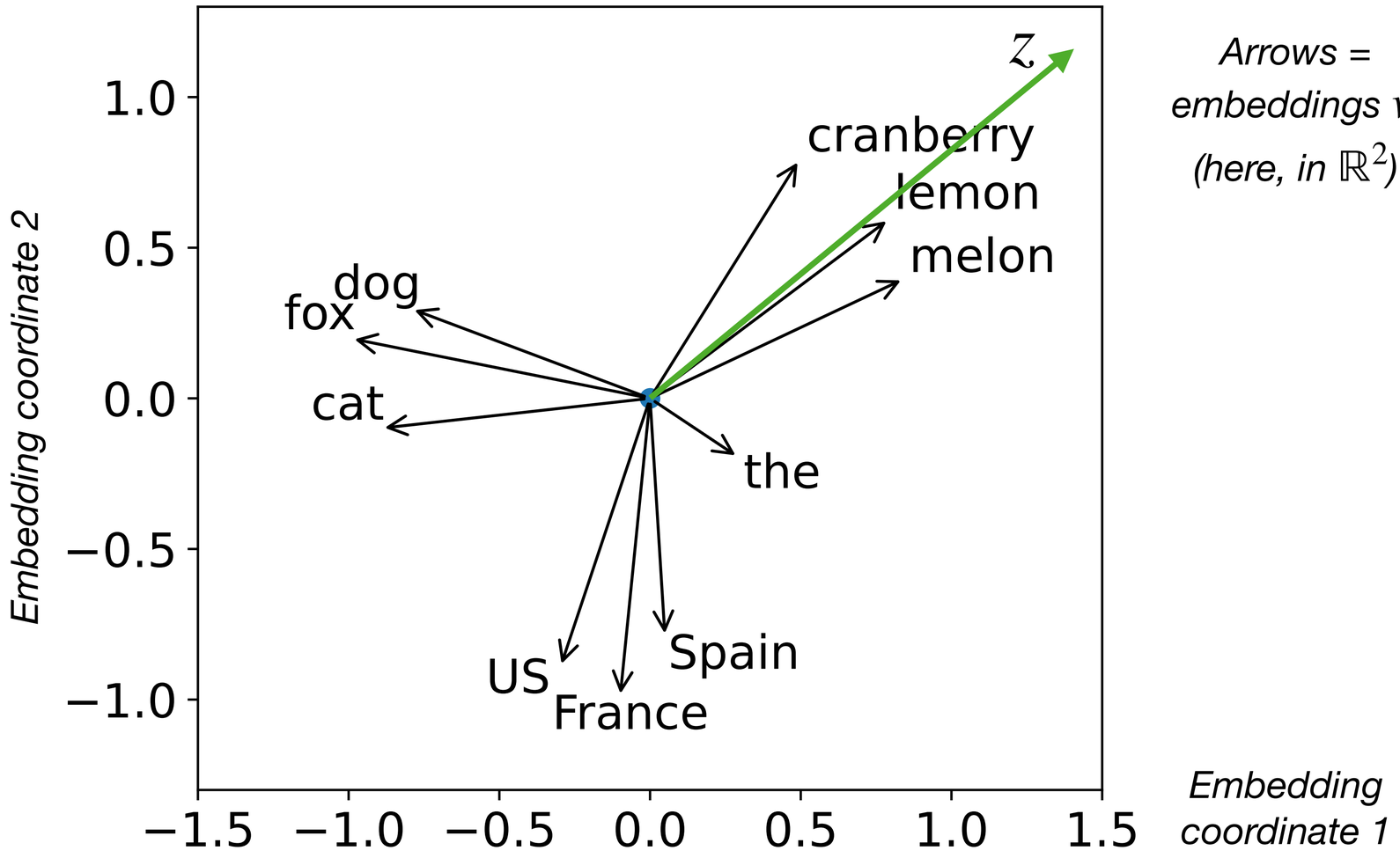


Arrows = embeddings v_j (here, in $\mathbb{R}^2$)

prediction task: … cranberry [???] melon …

- z = average of context tokens · window length

First layer

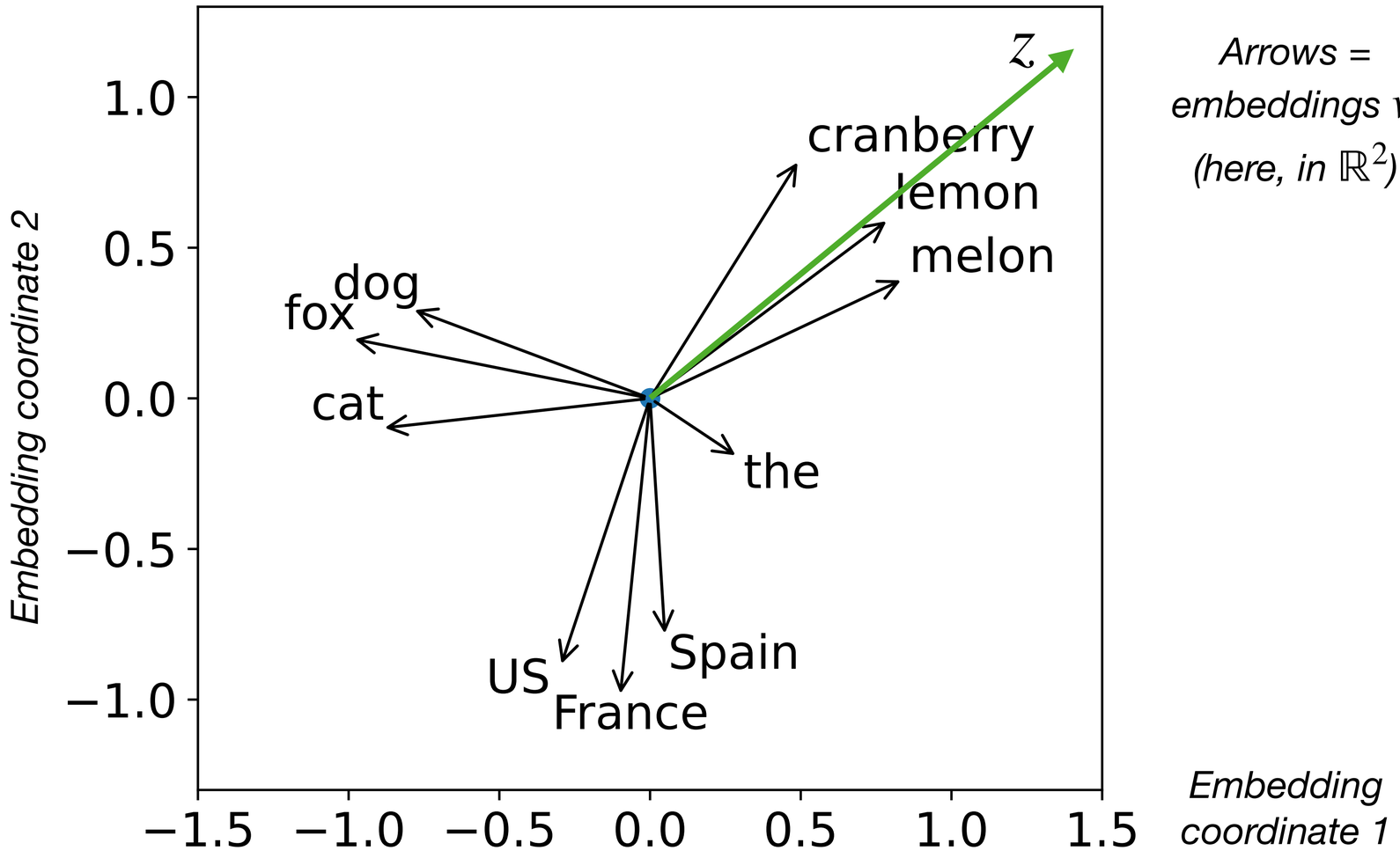


Arrows = embeddings v_j (here, in $\mathbb{R}^2$)

prediction task: … cranberry [???] melon …

- z = average of context tokens · window length

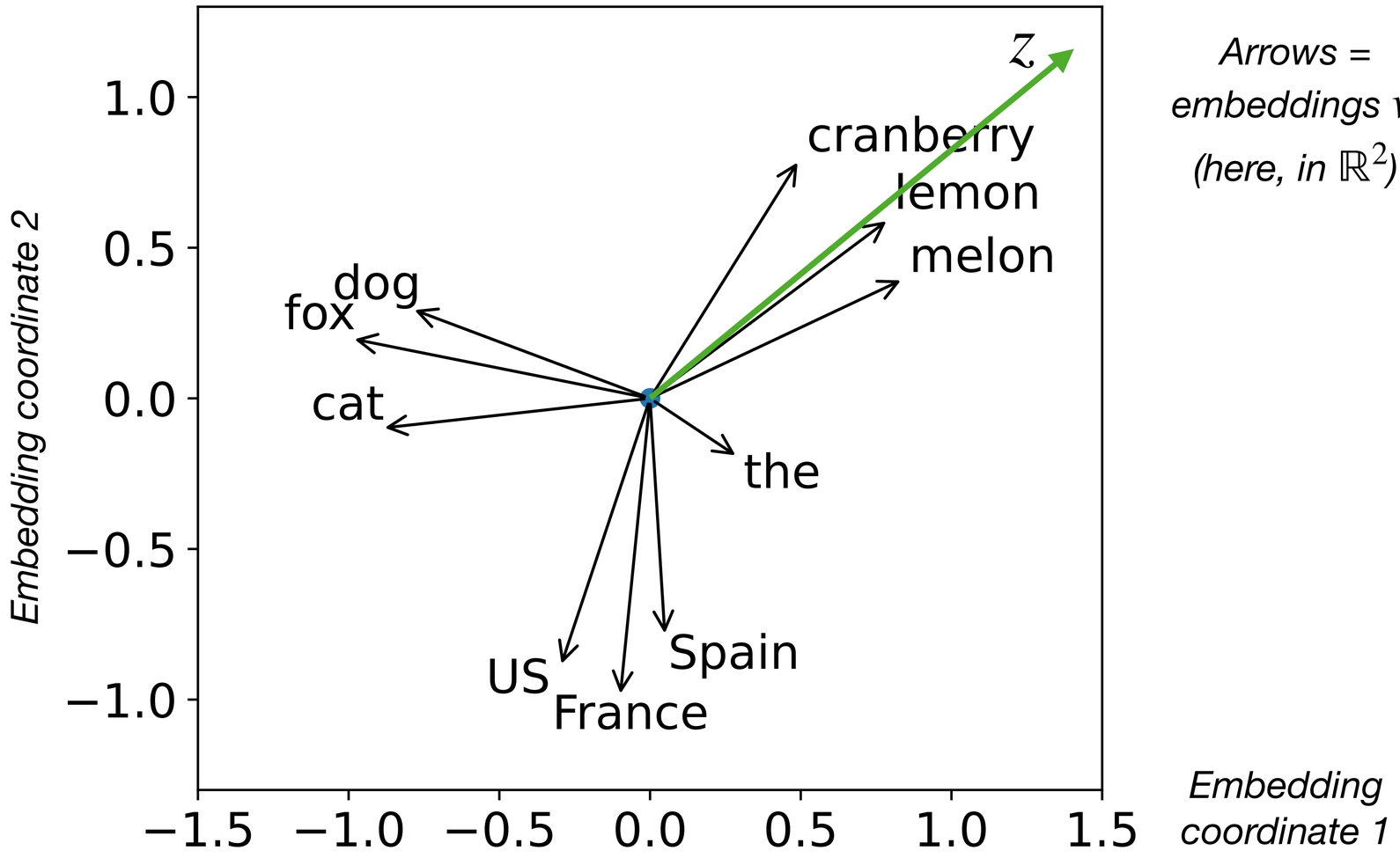
Second layer



Arrows = embeddings v_j (here, in $\mathbb{R}^2$)

- Input to second layer = a vector z, summary of context
- Goal of second layer: search through a set of vectors (the vocabulary) to find similar ones to context

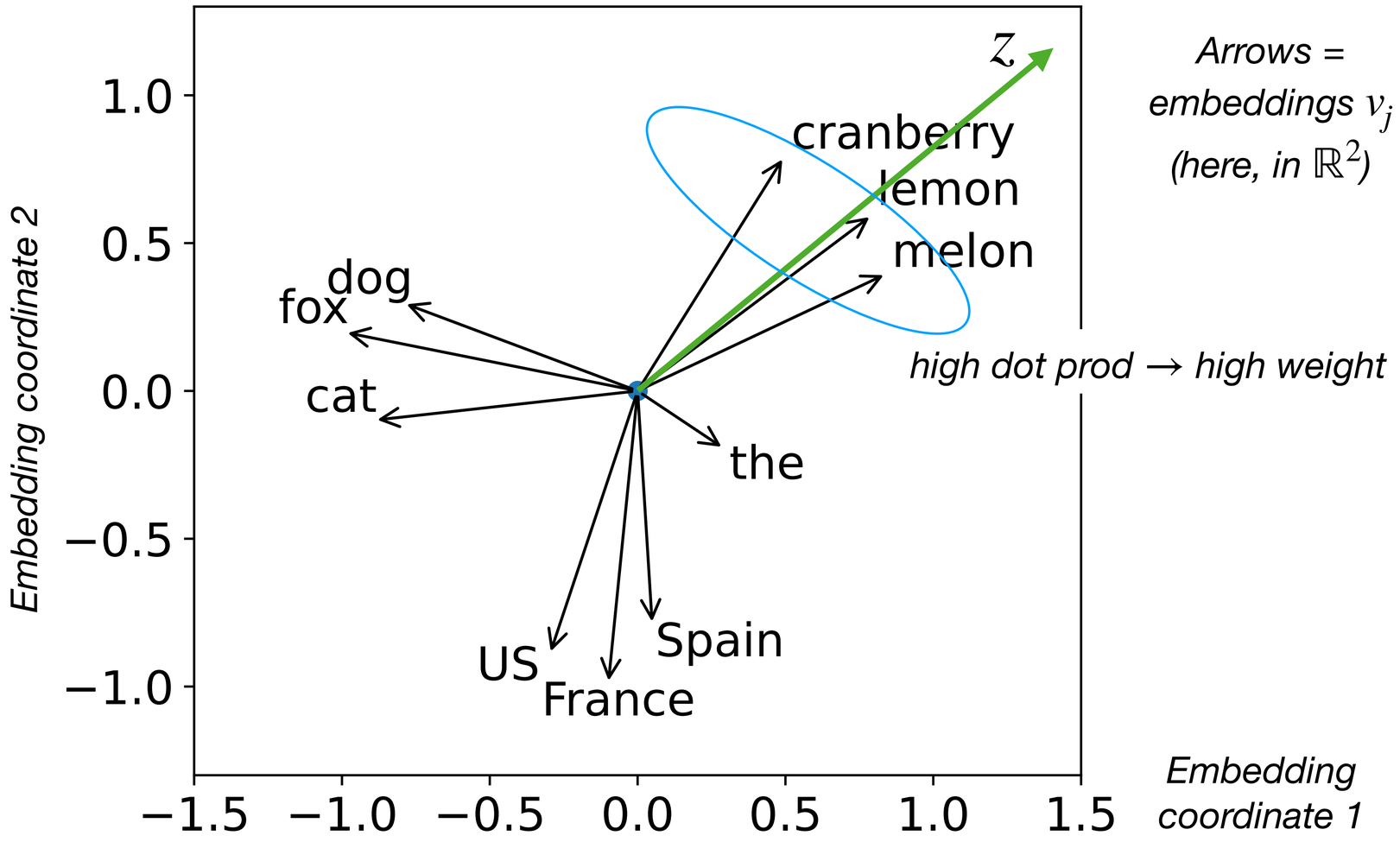
Second layer



Arrows = embeddings v_j (here, in $\mathbb{R}^2$)

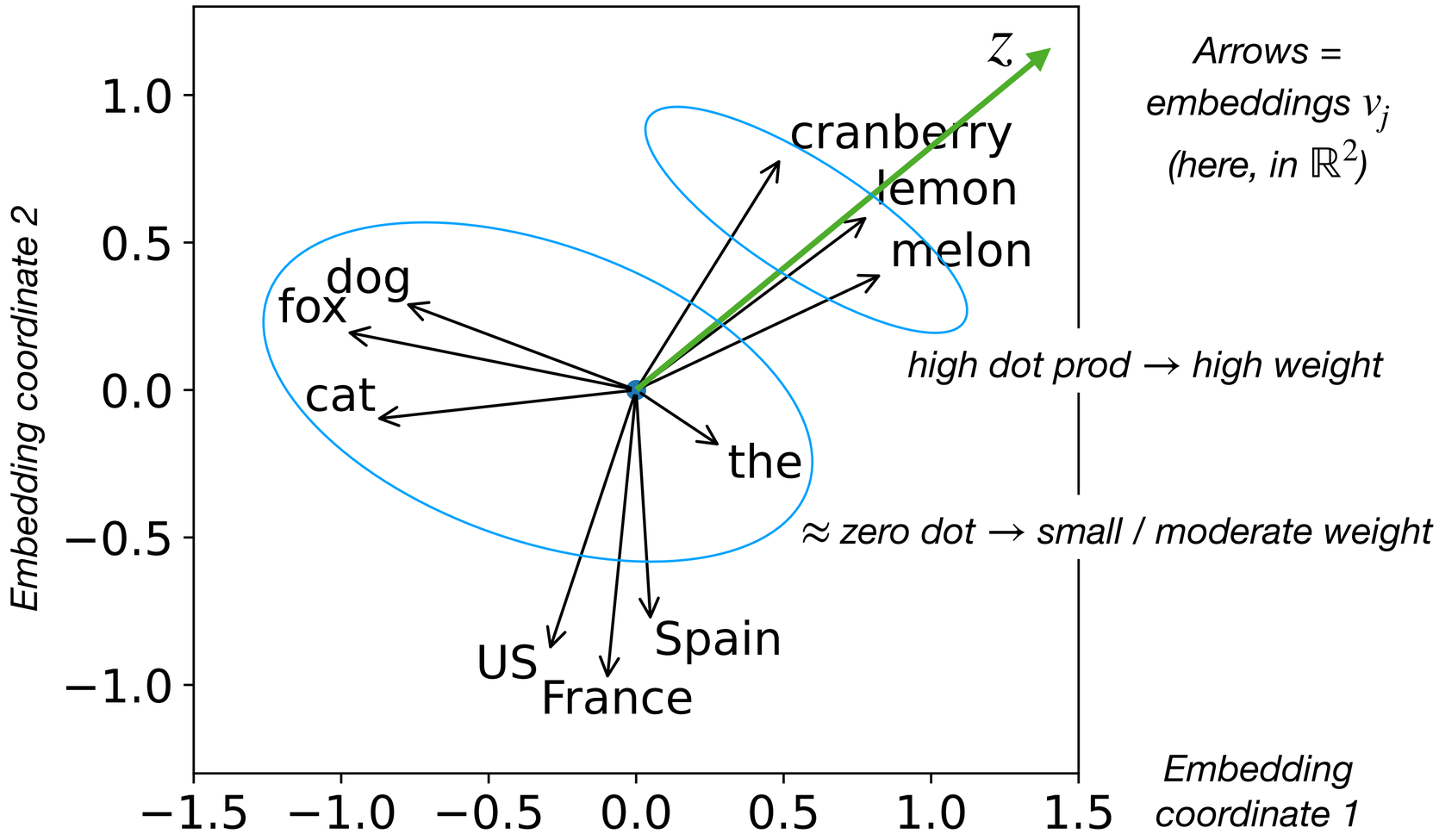
- Dot product $v_j \cdot z$ is highest for token embeddings that are both *high norm* and *similar direction* to z
- The $\exp$ in softmax further emphasizes highest values

Second layer



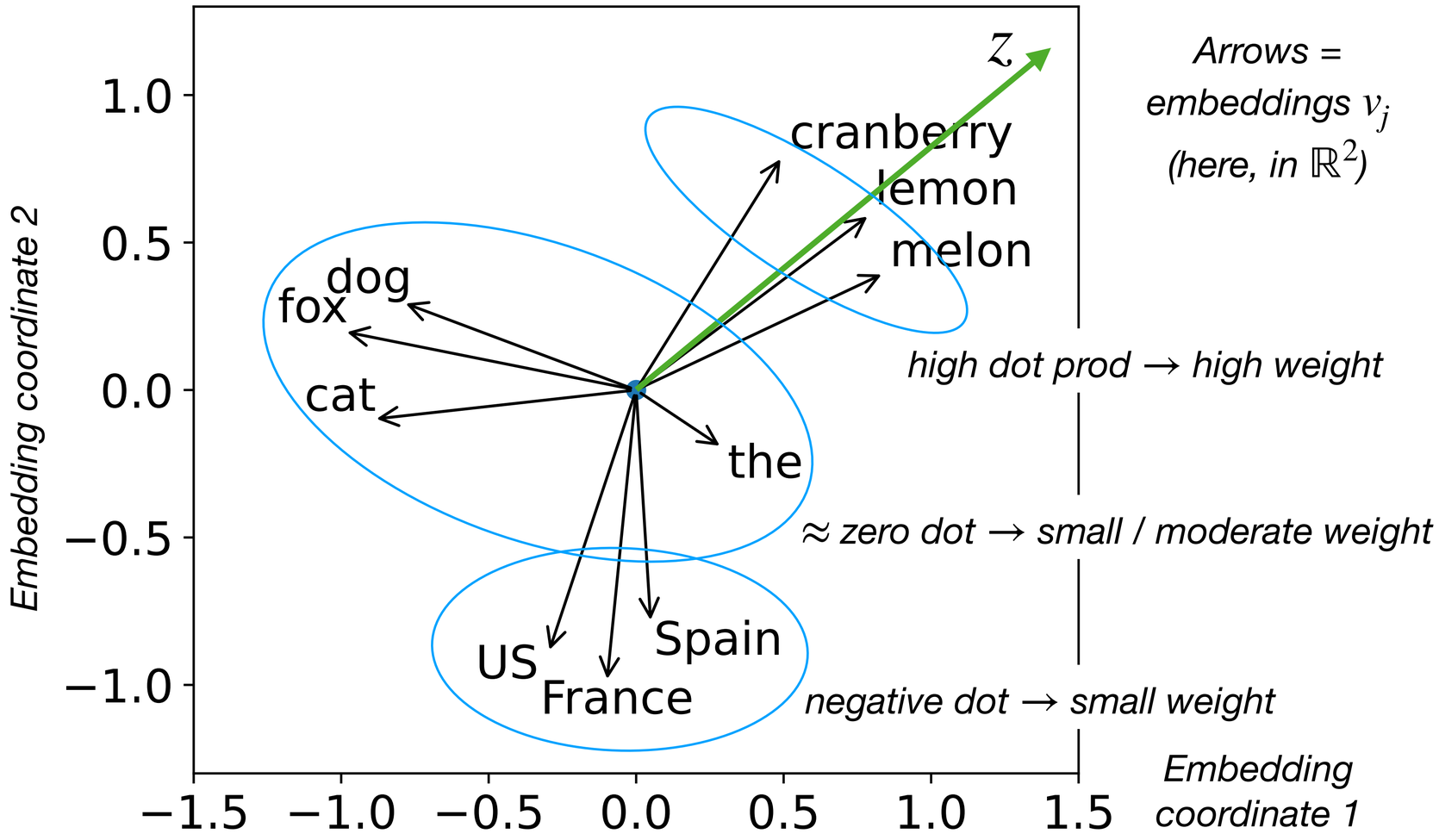
- Dot product $v_j \cdot z$ is highest for token embeddings that are both *high norm* and *similar direction* to z
- The $\exp$ in softmax further emphasizes highest values

Second layer



- Dot product $v_j \cdot z$ is highest for token embeddings that are both *high norm* and *similar direction* to z
- The $\exp$ in softmax further emphasizes highest values

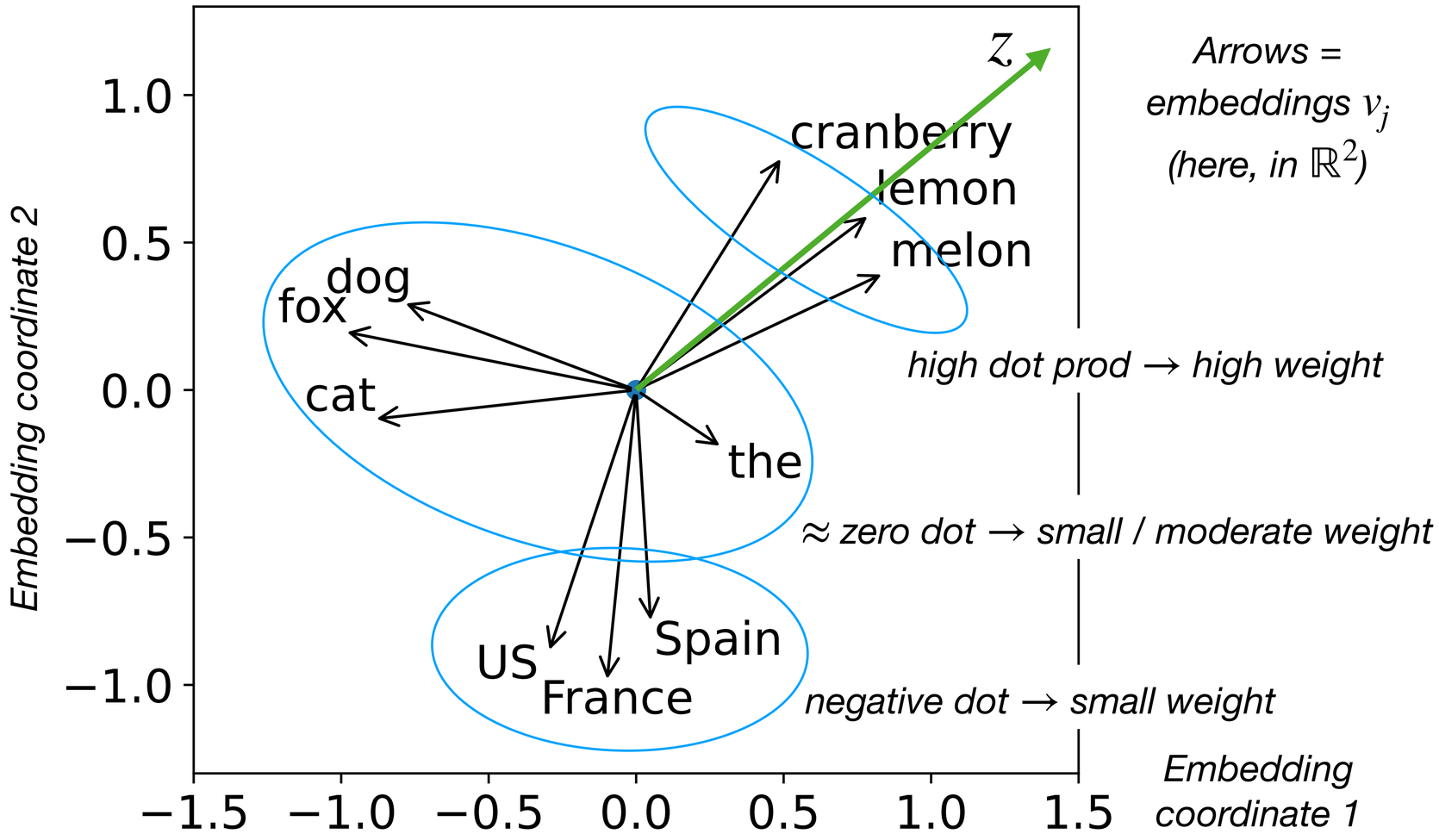
Second layer



- Dot product $v_j \cdot z$ is highest for token embeddings that are both *high norm* and *similar direction* to z
- The $\exp$ in softmax further emphasizes highest values

Second layer

*This use of softmax (to find vocabulary words that have high dot product w/ context) is strongly related to **attention** (coming up soon)*

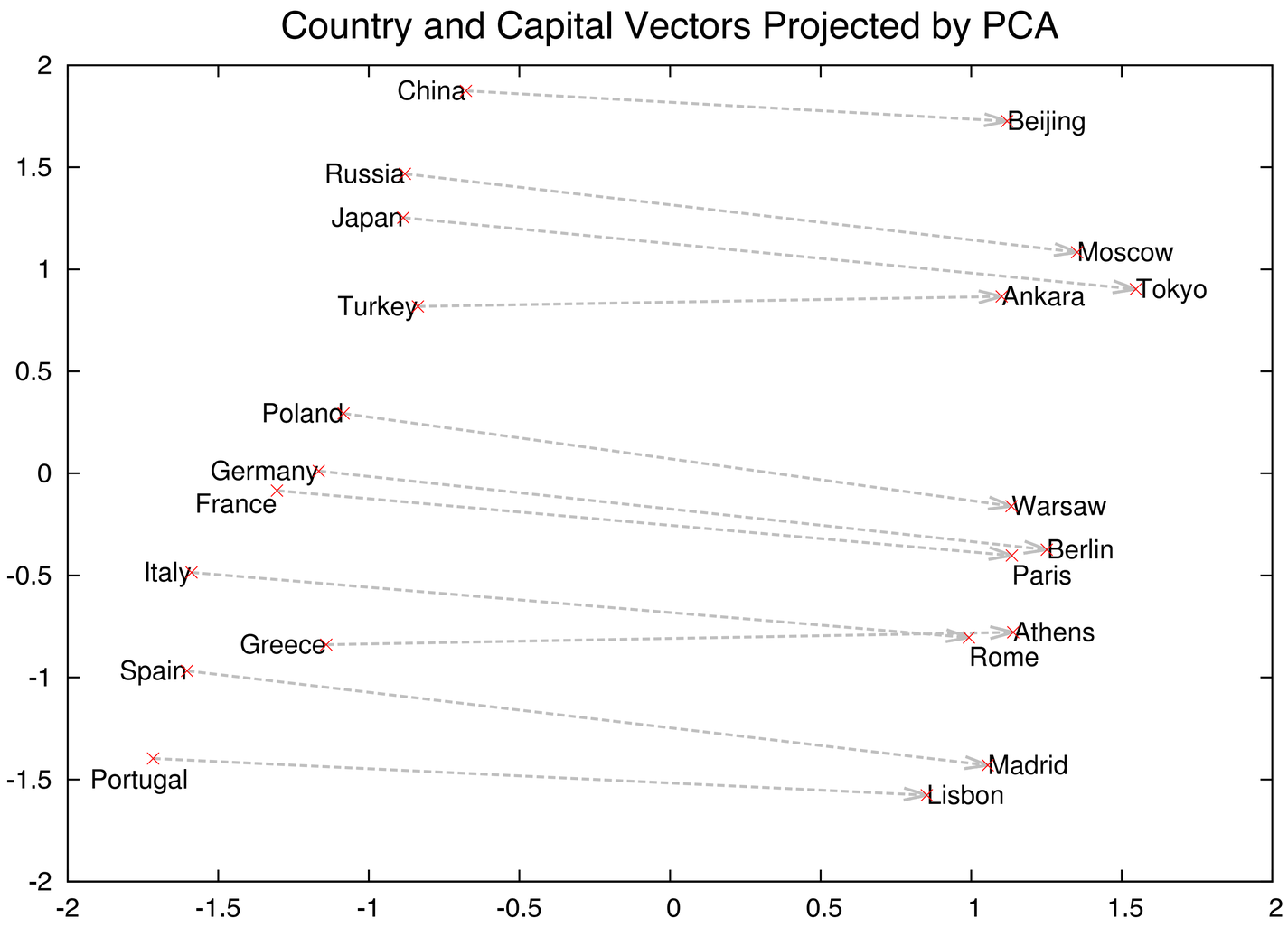


- Dot product $v_j \cdot z$ is highest for token embeddings that are both *high norm* and *similar direction* to z
- The $\exp$ in softmax further emphasizes highest values

SGD step

- Each optimization step will try to
  - shift embedding of (words in the) query to better retrieve correct masked word (put more attention weight on it)
  - shift the correct masked word's embedding to better match query (have higher dot product with it)
  - shift all other words' embeddings to have lower dot products with the query

Some learned embeddings



- A 2d projection of some learned vector-space embeddings from word2vec

Mikolov et al., https://arxiv.org/abs/1310.4546

Learning goals: deep nets

- What is a deep neural network?
- What are depth, width, and work, and why do they matter?
- What are common elements/blocks of deep nets? For each one, why might you include it in a network?
- How do you calculate the loss and its gradient, given a training example (x, y)?
- How can you set up and use autodiff in a library like PyTorch?
- What are vanishing or exploding gradients? Why are they a problem? How can we mitigate this problem?

Learning goals: CNNs

- What is a tensor, and what are common tensor operations?
- What are Einstein summation and tensor contraction?
- What is a (continuous time or space) signal? How does it relate to a (discrete time or space) sampled vector, matrix, or tensor?
- What is a convolution? What tasks can it accomplish?
- What variations exist of the convolution operation?
- What are equivariance and invariance, and how do we implement them in a CNN?
- What are multi-head convolutions, layerwise convolutions, channels, pooling, vec/unvec?

Learning goals: RNNs

- What different kinds of sequence prediction tasks might we encounter and how can we present them to an RNN?
- What does a basic RNN block look like, and how do we use it to make predictions?
- What is a vector-space embedding, and what is it used for?
- What is long-term memory in an RNN, how does it relate to gating, and what problem does it solve?
- What is backprop through time? How is it similar or different to backprop in other deep nets?
- How can we train an RNN language model? How do we use it to predict?